Facultad de Ingeniería y Arquitectura

Teoría de aprendizaje de máquina

Julio 20- 2025

Diagnóstico Automatizado de Fallos en Rodamientos Mediante Aprendizaje Automático Aplicado a Señales de Vibración del Dataset CWRU

Por: Julián Felipe Gutiérrez Ramírez, Samuel Marulanda Salazar y Felipe Idárraga Quintero

Fecha de presentación ***20/07/2025***

Resumen

El presente informe describe el desarrollo de un sistema inteligente para la detección automática de fallos en rodamientos de motores eléctricos, utilizando técnicas de aprendizaje automático y aprendizaje profundo. Se empleó el conjunto de datos CWRU, con características estadísticas extraídas de señales de vibración. El sistema fue entrenado con tres modelos: Máquina de Vectores de Soporte (SVC), Bosque Aleatorio (RF) y Perceptrón Multicapa (MLP). Se incluye el flujo de preprocesamiento, las métricas de evaluación y una interfaz interactiva implementada en Streamlit que permite a usuarios no técnicos realizar diagnósticos en tiempo real.

Palabras clave: Diagnóstico de fallos, rodamientos, aprendizaje automático, aprendizaje profundo, SVC, Random Forest, MLP, Streamlit, CWRU.

Abstract

This report presents the development of an intelligent system for the automatic detection of bearing faults in electric motors, using machine learning and deep learning techniques. The CWRU dataset was used, containing statistical features extracted from vibration signals. The system was trained with three models: Support Vector Machine (SVC), Random Forest (RF), and Multilayer Perceptron (MLP). The report includes the preprocessing pipeline, evaluation metrics, and an interactive Streamlit interface that enables non-technical users to perform real-time diagnostics.

Keywords: Fault diagnosis, bearings, machine learning, deep learning, SVC, Random Forest, MLP, Streamlit, CWRU

1. Motivación del Proyecto

En entornos industriales, los motores eléctricos juegan un papel fundamental en la operación de sistemas rotativos, siendo los rodamientos uno de sus componentes más críticos. La presencia de fallos no detectados en estos elementos puede derivar en paradas imprevistas, daños en maquinaria, pérdida de productividad y riesgos significativos para la seguridad del personal. Por lo tanto, la detección temprana de anomalías en rodamientos se ha convertido en un objetivo prioritario dentro de las estrategias de mantenimiento industrial [1].

En este contexto, la aplicación de técnicas de aprendizaje automático (Machine Learning, ML) y aprendizaje profundo (Deep Learning, DL) ofrece una alternativa eficiente y precisa frente a los métodos tradicionales de monitoreo. Estas metodologías permiten analizar grandes volúmenes de datos provenientes de sensores de vibración, identificar patrones anómalos y anticipar fallos incipientes con un alto grado de confiabilidad [2]. Su implementación favorece la transición hacia esquemas de mantenimiento predictivo, disminuyendo la dependencia del mantenimiento correctivo o preventivo programado [3].

La iniciativa se justifica desde diversas dimensiones. Técnicamente, se promueve el desarrollo de sistemas inteligentes capaces de operar en tiempo real y con capacidad de generalización ante distintos tipos de fallas. Económicamente, se contribuye a la reducción de costos operativos al minimizar el tiempo de inactividad y evitar daños severos. Socialmente, se mejora la seguridad en el entorno laboral al prevenir incidentes derivados de fallos inesperados. Finalmente, científicamente, el proyecto representa una oportunidad para avanzar en la investigación aplicada de modelos ML/DL en el ámbito del monitoreo de condición industrial [4].

2 Planteamiento del Problema

Los rodamientos son componentes esenciales en maquinaria rotativa industrial, y su falla representa una de las principales causas de interrupciones no planificadas, reducción de la eficiencia operativa y costos elevados de mantenimiento correctivo [1]. Si bien existen técnicas tradicionales de monitoreo como el análisis espectral o las inspecciones manuales, estas presentan limitaciones en cuanto a precisión, velocidad de diagnóstico y sensibilidad ante ruido o condiciones cambiantes [6].

En consecuencia, surge la necesidad de desarrollar soluciones automatizadas que, mediante el análisis de señales de vibración, permitan detectar de manera temprana y confiable las fallas incipientes en rodamientos. Para ello, las técnicas de aprendizaje automático (ML) y aprendizaje profundo (DL) han demostrado ser herramientas eficaces para la extracción de características, la clasificación de condiciones operativas y la predicción del estado de salud de componentes mecánicos [4].

Pregunta de investigación:
¿Es posible desarrollar un modelo basado en técnicas de ML/DL que permita detectar y clasificar fallos en rodamientos de motores eléctricos a partir de señales de vibración, con alta precisión y robustez frente a ruido e incertidumbre operativa?

Este problema se aborda en el contexto del mantenimiento predictivo en la industria, con el objetivo de anticipar fallos antes de que se produzcan consecuencias severas, contribuyendo así a la optimización de procesos y a la mejora de la seguridad industrial.

Conjunto de datos y dominio de aplicación:
La solución propuesta se aplicará al monitoreo de condición de motores eléctricos mediante el uso **del Case Western Reserve University Bearing Data Center dataset (CWRU)** [5], uno de los repositorios más utilizados en investigaciones sobre diagnóstico de fallos. Este conjunto de datos contiene señales de vibración capturadas por acelerómetros montados sobre rodamientos bajo diversas condiciones de operación (normales y defectuosas), velocidades de rotación y niveles de carga. Su riqueza y diversidad permiten entrenar y evaluar modelos de clasificación supervisada con alta representatividad industrial.

3. Estado del Arte:

En esta sección se presentan **tres trabajos representativos** que emplean el conjunto de datos CWRU y técnicas de ML/DL para el diagnóstico de fallos en rodamientos. Se describen sus enfoques, fortalezas.

Tabla 1. Resumen de trabajos representativos para el conjunto de datos CWRU

Ref	Enfoque	Fortalezas
[7] Zhang *et al.* (IEEE Access, 2021) – *Review sobre ML con CWRU*. Revisión exhaustiva de más de 80 estudios que usan CWRU. Clasifica enfoques según dominio de características (temporal, frecuencial, tiempo-frecuencia) y técnicas (SVM, KNN, ANN, RF, CNN).	- Amplia cobertura teórica. - Identifica retos comunes como el sobreajuste o selección sesgada de ventanas. - Resume rendimiento comparativo de múltiples modelos.	- No propone un sistema propio. - Solo cubre el dataset CWRU. - No analiza desempeño en tiempo real ni eficiencia computacional.
[8] Raj *et al.* (MDPI Information, 2024) – *Enhanced Fault Detection…*. Se entrena una MLP directamente con datos crudos de acelerómetros sin transformación espectral previa. Evalúa desempeño con múltiples métricas y destaca la ventaja de usar señales no procesadas.	- Elimina la necesidad de ingeniería de características. - MLP simple, entrenado eficientemente. - Métricas detalladas (accuracy, precisión, recall, F1). Clásicos.	- Limitado a CWRU. - No incluye comparación con redes convolucionales. - No aborda ruido externo ni escalabilidad.
[9] Hang Trance (Kaggle Notebook, 2023) – *CWRU Bearings Fault Classification*. Implementa un pipeline reproducible usando modelos clásicos: **Random Forest** y **SVC**. Realiza EDA, extracción FFT, codificación de etiquetas y evaluación con matriz de confusión.	- Fácil de replicar y modificar. - SVC y RF alcanzan buena precisión con bajo costo computacional. - Buen material para benchmarking rápido.	- Selección manual de hiperparámetros. - División aleatoria puede inducir *data leakage*. - Dataset limitado sin validación cruzada.

Síntesis crítica.

- El trabajo [7] proporciona un marco teórico sólido que orienta la selección de técnicas y revela tendencias comunes.
- [8] demuestra que es posible entrenar una MLP directamente con datos crudos, lo que simplifica la cadena de procesamiento sin comprometer el rendimiento, aunque aún debe validarse en entornos con ruido industrial.
- [9] evidencia que modelos clásicos como SVC y Random Forest siguen siendo competitivos, especialmente cuando se dispone de bajo poder computacional, pero con riesgo de sobreajuste si no se gestionan correctamente las particiones y los hiperparámetros.

En conjunto, estos estudios fundamentan la elección de técnicas y la necesidad de diseñar un sistema robusto, interpretable y validado en condiciones cercanas a las industriales.

4. Objetivo del Proyecto

Objetivo general:

Desarrollar y evaluar un sistema computacional basado en técnicas de aprendizaje automático y aprendizaje profundo para

la detección y clasificación automática de fallos en rodamientos de motores eléctricos, utilizando como insumo las características derivadas del conjunto de datos CWRU.

Objetivos específicos:

- Implementar un pipeline de procesamiento que permita la lectura, análisis y preparación de las señales de vibración disponibles en la base de datos CWRU, facilitando su uso en tareas de clasificación supervisada.
- Entrenar y comparar modelos de aprendizaje automático (como Support Vector Machines y Random Forest) y modelos de aprendizaje profundo (como Perceptrones Multicapa, MLP), evaluando su rendimiento en términos de precisión, eficiencia computacional y capacidad de generalización.

5. Metodología Propuesta

Para abordar el problema de detección y clasificación de fallos en rodamientos de motores eléctricos, se diseñó un flujo de trabajo estructurado que comprende las siguientes etapas:

- ✓ **Recolección y preprocesamiento de datos**
  Se utilizó el archivo feature_time_48k_2048_load_1.csv, proveniente del conjunto de datos de CWRU disponible en Kaggle [10]. Este archivo contiene características estadísticas de señales de vibración registradas a 48 kHz bajo condiciones de carga constante. Durante el preprocesamiento:
  - Se verificó que no existieran datos faltantes.
  - Se codificaron las etiquetas correspondientes a las condiciones del rodamiento.
  - Se aplicó normalización Z-score mediante StandardScaler para estandarizar los atributos numéricos.

- ✓ **División de datos en entrenamiento y prueba:**
  El conjunto fue dividido de forma estratificada en dos subconjuntos:
  - Entrenamiento (n = total − 750)
  - Prueba (n = 750)

  Esta división asegura una adecuada representatividad de las clases en ambos subconjuntos y evita el sesgo de muestreo.

- ✓ **Selección de algoritmos o arquitecturas**
  Se implementaron tres modelos de clasificación supervisada:
  - Support Vector Machine (SVC)
  - Random Forest (RF)
  - Perceptrón Multicapa (MLP) desarrollado en PyTorch

  La elección de estos modelos se basó en trabajos previos [7]–[9] y en su capacidad de balancear rendimiento, interpretabilidad y eficiencia computacional.

- ✓ **Métricas de evaluación**
  El rendimiento se evaluó en el conjunto de prueba usando:
  - Precisión (accuracy)
  - F1-score macro y por clase
  - Matriz de confusión
  -

- ✓ **Estrategias de mejora del rendimiento**
  Para optimizar el desempeño y facilitar la interacción del usuario final, se implementaron las siguientes estrategias:
  - Ajuste manual de hiperparámetros en los modelos
  - Regularización del MLP con técnicas como dropout.
  - Desarrollo de una aplicación interactiva en Streamlit que permite a cualquier persona:
    - ➢ Ingresar manualmente las características del motor (correspondientes al CSV utilizado)
    - ➢ Elegir el modelo de clasificación (MLP, SVC o Random Forest)
    - ➢ Obtener la predicción del estado del motor
    - ➢ Visualizar las probabilidades de clasificación por clase

  Esta App hace accesible el sistema de diagnóstico incluso para usuarios no técnicos, facilitando la adopción de tecnologías de aprendizaje automático en entornos industriales.

6. Background de los Modelos Utilizados

En este proyecto se han seleccionado tres algoritmos de clasificación supervisada: Support Vector Machine (SVC), Random Forest (RF) y Perceptrón Multicapa (MLP). Cada uno aporta ventajas particulares para abordar el diagnóstico de fallas en rodamientos a partir de características tabulares extraídas de señales de vibración.

- **Support Vector Machine (SVC)**
  Las máquinas de vectores de soporte son clasificadores robustos que buscan encontrar el hiperplano que mejor separa las clases en un espacio de características de alta dimensión. Utilizan funciones kernel (como RBF o polinomial) para transformar los datos no linealmente separables en espacios donde sí lo son. SVC es

especialmente útil en contextos con:

- Tamaños de muestra moderados
- Dimensiones elevadas
- Clases bien definidas

En este caso, el modelo SVC permite una clasificación precisa entre tipos de fallas mediante un margen óptimo, mostrando buen rendimiento en estudios previos sobre el conjunto CWRU [7], [12].

- **Random Forest (RF)**
  El algoritmo Random Forest es un modelo basado en árboles de decisión, que construye múltiples árboles sobre subconjuntos aleatorios de los datos y promedia sus predicciones. Esta técnica reduce el sobreajuste y mejora la generalización. Sus ventajas clave incluyen:
  - Capacidad de manejar características correlacionadas
  - Robustez ante ruido
  - Interpretabilidad mediante importancia de variables

  En el contexto del diagnóstico de fallas, RF es útil para capturar relaciones no lineales complejas entre las características estadísticas sin necesidad de normalización estricta [9].

- **Perceptrón Multicapa (MLP)**
  El MLP es una red neuronal artificial feedforward compuesta por capas densamente conectadas (input, ocultas y salida), que se entrena mediante retro propagación. En este trabajo, se usó un MLP diseñado con PyTorch, entrenado sobre los vectores normalizados de características. Sus ventajas incluyen:
  - Capacidad de aprendizaje universal para funciones no lineales
  - Adaptabilidad a diferentes distribuciones de entrada
  - Buen desempeño en clasificación multiclase con datos estructurados

  Dado que los datos ya están representados como vectores de características numéricas, el uso de un MLP resulta adecuado y computacionalmente eficiente, evitando la necesidad de arquitecturas convolucionales más complejas [8].

7. Experimental Set-Up

Esta sección describe el entorno computacional, las librerías utilizadas, el tamaño del dataset y el diseño detallado del pipeline experimental utilizado para entrenar, evaluar y desplegar modelos de clasificación de fallas en rodamientos.

✓ **Software y librerías utilizadas**

El desarrollo se realizó en Python 3.10 y se utilizaron las siguientes librerías:

- Procesamiento de datos y EDA: pandas, numpy, gdown, pathlib, joblib
- Visualización: matplotlib, seaborn, IPython.display
- Modelado y evaluación (ML): scikit-learn (SVC, RandomForestClassifier, Pipeline, StandardScaler, LabelEncoder, train_test_split, GridSearchCV, classification_report, confusion_matrix, accuracy_score)[11], xgboost
- Modelado (DL): torch, torch.nn, TensorDataset, DataLoader
- Aplicación web: streamlit, pyngrok

Estas herramientas fueron seleccionadas por su robustez, facilidad de integración y comunidad activa, permitiendo construir un flujo completo desde el análisis exploratorio hasta el despliegue del modelo.

✓ **Especificaciones del hardware**

- CPU: Intel Core i5-10300H @ 2.50GHz
- RAM: 16 GB DDR4
- SO: Windows 11 Pro-64-bit
- GPU: No se usó aceleración GPU
- Entornos de desarrollo: Google Colab.

✓ **Tamaño de los datos**

Se trabajó con el archivo **feature_time_48k_2048_load_1.csv** de Kaggle, que contiene **8085 muestras**. Cada muestra representa una condición del motor eléctrico y está compuesta por un vector de **características estadísticas ya preprocesadas** (por ejemplo: media, varianza, RMS, skewness, curtosis) y su respectiva etiqueta de clase.

✓ **Detalles del pipeline experimental**

A. Preprocesamiento de datos

- Se verificó que el conjunto no contenía valores nulos.
- Se utilizó LabelEncoder para convertir las etiquetas de clase (estado del rodamiento) en valores numéricos.
- Se aplicó **normalización Z-score** a los atributos con StandardScaler, lo cual es esencial para modelos como SVC y MLP que son sensibles a la escala de las variables.

B. División del conjunto de datos

- Se realizó una **partición estratificada** usando train_test_split con test_size=750.
- Esta división mantuvo la proporción original de clases en ambos subconjuntos.

C. Entrenamiento de modelos

Se entrenaron tres modelos con pipelines específicos:

- **Random Forest Classifier**: Pipeline: StandardScaler → RandomForestClassifier Entrenado con un número limitado de árboles y profundidad máxima para evitar sobreajuste.

- **Support Vector Classifier (SVC)**: Pipeline: StandardScaler → SVC(kernel='rbf') Se probó con diferentes valores de C y gamma mediante GridSearchCV.
- **Multilayer Perceptron (MLP)**: Implementado con PyTorch. Arquitectura:
    - Input layer: igual al número de características
    - 1 o 2 capas ocultas con activación ReLU
    - Output layer con softmax
    - Optimización con Adam, CrossEntropyLoss, y entrenamiento durante varias épocas

D. Evaluación del desempeño

Los modelos fueron evaluados en el conjunto de prueba utilizando:

- **Accuracy**: para conocer el rendimiento global
- **F1-score macro**: para comparar clases balanceadas y desbalanceadas
- **Matriz de confusión**: para identificar tipos de errores
- **Curvas ROC y AUC**: cuando fue posible (en clasificación binaria o multiclase one-vs-rest)

E. Despliegue de la aplicación (Streamlit)

Se desarrolló una interfaz interactiva en **Streamlit** donde el usuario puede:

- Ingresar manualmente un conjunto de características
- Elegir el modelo de clasificación (MLP, SVC, RF)
- Obtener la predicción del estado del motor
- Visualizar la **probabilidad de pertenencia a cada clase**

Además, se integró con **Pyngrok** para hacer pública la app desde un entorno local

8. Resultados y Discusión

Los modelos fueron evaluados sobre un conjunto de prueba de **750 muestras** distribuidas en **10 clases** balanceadas (75 muestras por clase). A continuación, se resumen los resultados cuantitativos obtenidos.

Tabla 2. Desempeño comparativo de modelos de clasificación (Accuracy y F1-Score)

Modelo	Accuracy (%)	Macro F1-Score	Weighted F1-Score
Random Forest	96.53	0.96	0.96
SVC	96.40	0.96	0.96
MLP (PyTorch)	93.07	0.93	0.93

Ambos modelos clásicos (Random Forest[12] y SVC [13]) superaron el 95% de precisión en el conjunto de prueba. El MLP también obtuvo un rendimiento sólido, aunque ligeramente inferior.

Tabla 3: Rendimiento por clase en el conjunto de prueba (F1-Score por modelo)

Clase	SVC	RF	MLP
Ball_007_1	0.97	0.97	0.93
Ball_014_1	0.91	0.89	0.86
Ball_021_1	0.93	0.88	0.83
IR_007_1	1.00	1.00	1.00
IR_014_1	1.00	1.00	0.99
IR_021_1	1.00	1.00	0.94
Normal_1	0.99	1.00	0.97
OR_007_6_1	1.00	1.00	1.00
OR_014_6_1	0.86	0.88	0.75
OR_021_6_1	0.99	0.97	0.98

- Todas las clases relacionadas con fallos en el anillo interior (IR) fueron clasificadas perfectamente por todos los modelos.
- Las clases más difíciles de predecir fueron OR_014_6_1 y Ball_021_1, con menor F1 en MLP y SVC. Esto sugiere mayor solapamiento entre sus representaciones estadísticas.
- El modelo Random Forest demostró una consistencia superior en casi todas las clases.

Fortalezas y limitaciones:

Fortalezas:

- Random Forest y SVC muestran excelente rendimiento sin necesidad de arquitecturas profundas.[12], [13].
- La normalización con Z-score fue efectiva para SVC y MLP.
- El modelo MLP, aunque más sensible al tuning, logra buenos resultados sin arquitectura compleja.
- La app Streamlit permitió poner a prueba los modelos en un entorno simulado de usuario final.

Limitaciones:

- Todos los modelos fueron evaluados únicamente en el dataset CWRU, por lo que su capacidad de generalización a entornos industriales reales no ha sido verificada.
- No se aplicó validación cruzada para estimar la varianza de los resultados.
- La arquitectura del MLP y sus hiperparámetros podrían optimizarse aún más.

El desarrollo de una interfaz interactiva en Streamlit permitió validar la usabilidad del sistema. Usuarios no técnicos pueden ingresar características reales de un rodamiento, seleccionar el modelo y recibir retroalimentación inmediata. Este enfoque facilita la integración futura en procesos de mantenimiento predictivo, habilitando decisiones informadas sin depender de especialistas en datos.

Además, el hecho de que modelos clásicos superen el 96% de precisión indica que para conjuntos de datos tabulares bien definidos como CWRU, las soluciones tradicionales siguen siendo altamente competitivas y eficientes computacionalmente.

9. Conclusiones

Este proyecto logró desarrollar y evaluar un sistema computacional basado en técnicas de aprendizaje automático y profundo para la detección automática de fallos en rodamientos de motores eléctricos. A partir del conjunto de datos feature_time_48k_2048_load_1.csv del repositorio CWRU, se construyó un pipeline completo que incluyó preprocesamiento, entrenamiento de modelos, evaluación comparativa y despliegue interactivo mediante una aplicación web.
Los resultados obtenidos demuestran que tanto Random Forest como Support Vector Machine alcanzaron altos niveles de precisión (≥ 96%) y F1-score, superando al modelo MLP entrenado con PyTorch. Las métricas indicaron un comportamiento robusto en la mayoría de las clases, especialmente en fallos del anillo interior y condiciones normales. Además, la implementación de una interfaz en Streamlit permitió que el sistema sea accesible a usuarios no técnicos, mostrando no solo la predicción sino también las probabilidades asociadas a cada clase.
Se destacan como fortalezas del sistema:

- Uso efectivo de técnicas estándar (Z-score, estratificación, F1 macro).
- Comparación entre modelos clásicos y una red neuronal.
- Énfasis en la interpretabilidad y aplicabilidad práctica mediante una interfaz de usuario sencilla.

Aspectos que no se lograron o pueden mejorarse

- No se utilizó validación cruzada, lo cual limita la estimación de la varianza del rendimiento.
- No se incluyó un análisis del impacto del ruido, ni generalización a datos no vistos o provenientes de sensores en condiciones reales.
- El modelo MLP no fue optimizado exhaustivamente en términos de arquitectura e hiperparámetros.

Trabajo futuro

Como línea de trabajo futuro se propone:

- Evaluar el sistema en condiciones reales de planta, incorporando sensores físicos y ruido operativo para validar la robustez de los modelos.
- Explorar técnicas de aprendizaje no supervisado (como clustering) para escenarios sin etiquetado previo.
- Integrar el sistema a plataformas de monitoreo en tiempo real, mediante APIs o dispositivos embebidos (por ejemplo, Raspberry Pi o microcontroladores con edge computing).
- Optimizar el MLP mediante búsqueda bayesiana de hiperparámetros o redes más profundas con regularización avanzada.

10. Referencias

[1] R. B. Randall, *Vibration-based Condition Monitoring: Industrial, Aerospace and Automotive Applications*, Wiley, 2011.

[2] W. Zhang, G. Li, J. Zhang, and Z. Guo, "A deep convolutional neural network with new training methods for bearing fault diagnosis under noisy environment," *IEEE Access*, vol. 6, pp. 16929–16940, 2018.

[3] M. A. A. Hashem, K. R. Chowdhury, and M. A. H. Akhand, "Predictive maintenance of bearings using machine learning techniques with data fusion," in *Proc. IEEE Int. Conf. Industrial Informatics (INDIN)*, 2020, pp. 1–6.

[4] Y. Lei, N. Li, L. Guo, T. He, and J. Lin, "Machinery health prognostics: A systematic review from data acquisition to RUL prediction," *Mechanical Systems and Signal Processing*, vol. 104, pp. 799–834, 2018.

[5] Case Western Reserve University Bearing Data Center. [Online]. Available: https://engineering.case.edu/bearingdatacenter

[6] S. Yin, H. Luo, and S. X. Ding, "Real-time implementation of fault-tolerant control systems with performance optimization," *IEEE Transactions on Industrial Electronics*, vol. 61, no. 5, pp. 2402–2411, May 2014.

[7] X. Zhang, B. Zhao y Y. Lin, "Machine Learning Based Bearing Fault Diagnosis Using the Case Western Reserve University Data: A Review," *IEEE Access*, 2021, doi: 10.1109/ACCESS.2021.3128669.

[8] K. K. Raj, S. Kumar, R. R. Kumar, and M. Andriollo,

"Enhanced Fault Detection in Bearings Using Machine Learning and Raw Accelerometer Data: A Case Study Using the Case Western Reserve University Dataset," *Information*, vol. 15, no. 5, p. 259, 2024.

[9] Hang Trance, "CWRU Bearings Fault Classification," Kaggle Notebook, 2023. [Online]. Available: https://www.kaggle.com/code/hangtrance/crwu-bearings-fault-classification

[10] B. R. Japon, "CWRU Bearing Datasets," *Kaggle*, 2023. [Online]. Available: https://www.kaggle.com/datasets/brjapon/cwru-bearing-datasets

[11] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[12] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[13] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.